

C++ para maratona

Consulta rápida - Maratona CIn

Para lembrar comandos e escolher estratégias durante os estudos. Primeira vez no assunto? Comece pelo guia explicado: guias/guia-cpp.md. Esta versão de impressão complementa guias/consulta-rapida.md.

1. Começo de toda questão

```
#include <bits/stdc++.h> // Atalho do GCC; nao e header padrao.
using namespace std;    // cin em vez de std::cin.

int main() {            // O programa comeca aqui.
    ios::sync_with_stdio(false); // Desliga sincronizacao com stdio.
    cin.tie(nullptr);      // Desvincula cin do flush de cout.

    // Leia os dados, resolva e imprima a resposta.

    return 0;           // Termina normalmente.
}
```

Com essas configurações, use cin/cout de forma consistente. Em problemas interativos, faça o flush solicitado pelo protocolo.

Bibliotecas padrão: escolha conforme os comandos

Quero usar	Cabeçalho padrão
cin, cout	<iostream>
vector, string	<vector>, <string>
map, set, multiset	<map>, <set>
stack, queue, priority_queue	<stack>, <queue>
deque	<deque>
sort, min, max	<algorithm>
greater<T>	<functional>
numeric_limits<T>	<limits>

bits/stdc++.h costuma estar disponível com GCC, mas não faz parte do padrão C++. Os programas em templates/ usam os cabeçalhos padrão necessários.

2. Entrada, tipos e operações

Ler valores e imprimir resultados

```
int a, b;
cin >> a >> b;           // Precisa de >> entre cada variavel.
cout << a + b << '\n';    // Imprime e quebra a linha.
long long produto = 1LL * a * b; // Multiplica em long long.

string s;
cin >> s;                 // Uma palavra: para no espaco.
```

Uma linha inteira, depois de ler um número

```
cin.ignore(numeric_limits<streamsize>::max(), '\n');
getline(cin, s); // Le ate a quebra de linha, incluindo espacos.
```

ignore descarta o restante da linha anterior, incluindo a quebra de linha. Use `<limits>` e `<string>` além de `<iostream>`.

Tipos e operadores de consulta frequente

Sintaxe	O que faz
int, long long	Inteiros. Em juízes usuais: cerca de $\pm 2 \times 10^9$ e $\pm 9 \times 10^{18}$, respectivamente.
double	Número real aproximado; não representa todo decimal exatamente.
char c = '0';	Um caractere. O dígito zero é diferente do número 0.
bool ok = true;	Verdadeiro (true) ou falso (false).
x += y; x++;	Soma y a x; soma 1 a x.
a / b, a % b	Divisão inteira se ambos são inteiros; resto. Exigem b diferente de zero.
==, !=, <, <=	Igual, diferente, menor, menor ou igual.
&&, , !	E, ou, negação. && e usam curto-circuito.

Curto-circuito: proteja o acesso primeiro

```
if (!st.empty() && st.top() == x) {
    st.pop(); // top() so e avaliado se a pilha nao estiver vazia.
}
```

Os trechos pequenos deste PDF pressupõem que as variáveis já foram declaradas. Para programas completos com entrada e saída, abra templates/.

3. STL: vetor, pilha e filas

STL é a biblioteca de estruturas e algoritmos prontos de C++. Aqui, n é o tamanho da estrutura. Ler ou remover uma ponta exige que ela tenha elementos.

vector: posições que crescem no fim

Comando	Significado	Custo usual
<code>vector<int> v;</code>	Cria um vetor vazio.	$O(1)$
<code>v.push_back(7)</code>	Acrescenta 7 no fim.	$O(1)$ amortizado
<code>v.pop_back()</code>	Remove o último; não retorna valor.	$O(1)$
<code>v[i], v.at(i)</code>	Acessa índice; <code>at</code> verifica limites.	$O(1)$
<code>v.front(), v.back()</code>	Lê o primeiro ou o último.	$O(1)$
<code>v.resize(n)</code>	Ajusta o número de elementos.	$O(n)$, pior caso

```
vector<int> v(3); // Tres posicoes existentes: [0, 0, 0].
v[0] = 7;        // Preenche: [7, 0, 0].
v.push_back(9);  // Acrescenta: [7, 0, 0, 9].
```

Amortizado: somando muitas operações, o custo médio é constante. Um `push_back` isolado pode realocar o vetor e custar $O(n)$.

stack, queue e deque: qual ponta usar?

Estrutura	Comandos e significado
<code>stack<int> st;</code>	<code>push(x)</code> insere no topo; <code>top()</code> lê o topo; <code>pop()</code> remove o topo. O último a entrar é o primeiro a sair.
<code>queue<int> q;</code>	<code>push(x)</code> entra no fim; <code>front()</code> lê o início; <code>pop()</code> sai do início. O primeiro a entrar é o primeiro a sair.
<code>deque<int> d;</code>	<code>push_front(x)</code> / <code>push_back(x)</code> inserem nas pontas; <code>front()</code> / <code>back()</code> leem; <code>pop_front()</code> / <code>pop_back()</code> removem.

Essas operações custam $O(1)$ com os containers padrão. Em todas as estruturas deste guia, `.empty()` pergunta se está vazia e `.size()` dá a quantidade, em $O(1)$.

```
if (!st.empty()) {
    int saiu = st.top(); // Salve o valor ANTES de apagar.
    st.pop();           // pop() so remove: nao retorna o valor.
}
```

4. STL: chaves, prioridades e sort

Estrutura / comando	Significado e custo
<code>map<int,int> m;</code>	Associa uma chave a um valor, mantendo as chaves ordenadas.
<code>m[x]++</code>	Incrementa; cria chave com 0 se faltar. $O(\log n)$.
<code>m.find(x) / m.erase(x)</code>	Iterador da chave ou <code>end()</code> se faltar / remove a chave e seu valor. $O(\log n)$.
<code>set<int> s;</code>	<code>insert(x)</code> insere sem duplicatas; <code>count(x)</code> dá 0 ou 1; <code>erase(x)</code> remove x. $O(\log n)$.
<code>multiset<int> ms;</code>	<code>insert(x)</code> permite duplicatas. $O(\log n)$.
<code>ms.count(x) / ms.erase(x)</code>	Conta cópias / remove TODAS as cópias. $O(\log n + \text{cópias})$.
<code>priority_queue<int> pq;</code>	<code>push(x)</code> insere: $O(\log n)$ amortizado com vector padrão; <code>top()</code> lê o maior: $O(1)$; <code>pop()</code> remove o maior: $O(\log n)$.

```
auto it = ms.find(4);
if (it != ms.end()) ms.erase(it); // Remove UMA copia de 4.

priority_queue<long long, vector<long long>,
               greater<long long>> menor;
menor.push(8);
menor.push(3);
cout << menor.top(); // 3: heap minimo, acessa o menor.
```

A heap permite consultar uma prioridade; não é uma fila ordenada para percorrer. Para verificar uma chave no map sem criá-la, use `m.find(x) != m.end()`.

Ordenar: padrão crescente ou comparador próprio

```
sort(v.begin(), v.end()); // Crescente.
sort(v.begin(), v.end(), greater<int>()); // Decrescente.
```

Comparador para strings não vazias, sem espaços: defina a função fora de main.

```
bool compara(const string& a, const string& b) {
    return a + b < b + a; // Menor concatenacao lexicografica.
}
```

```
// Dentro de main(), depois de ler vector<string> palavras:
sort(palavras.begin(), palavras.end(), compara);
```

`begin()` aponta para o início; `end()` fica depois do último. O comparador responde: "o primeiro deve vir antes do segundo?" Use uma relação consistente e estrita; nunca `<=`. Para equivalentes, os dois sentidos devem dar false.

`sort` faz $O(n \log n)$ comparações. Concatenar strings custa $O(|a| + |b|)$ por comparação. `const string&` evita copiar o parâmetro e impede alterá-lo pela referência.

5. Estratégias: prefixos e janelas

Sinal no problema	Estratégia / custo
Muitas somas de intervalos; array não muda	Prefix sum: $O(n)$ preparo, $O(1)$ consulta.
Intervalos de exatamente k posições	Janela fixa: $O(n)$; com map, $O(n \log n)$.
Contar intervalos de soma x ; há negativos	Prefix sum + frequências: $O(n \log n)$ com map.
Manter escolhas, descartar a pior	Greedy + heap com justificativa: $O(n \log n)$.
Reordenar conforme uma regra	sort + comparador: $O(n \log n)$ comparações.

Prefix sum: soma inclusiva $[l, r]$, índices a partir de 1

```
vector<long long> pref(n + 1, 0);
for (int i = 0; i < n; ++i) {
    pref[i + 1] = pref[i] + a[i];
}
long long soma = pref[r] - pref[l - 1]; // 1 <= l <= r <= n.
```

$\text{pref}[i]$ soma os primeiros i elementos. Exemplo: $a = [2, 4, 1]$, $\text{pref} = [0, 2, 6, 7]$. Das posições 2 a 3: $7 - 2 = 5$. Para índices a partir de 0 e intervalo $[l, r)$, use $\text{pref}[r] - \text{pref}[l]$.

Janela fixa: tirar quem sai, somar quem entra

```
long long soma = 0;
for (int i = 0; i < k; ++i) soma += a[i];
// A primeira janela esta pronta: use soma aqui.
for (int r = k; r < n; ++r) {
    soma -= a[r - k]; // Sai o elemento mais antigo.
    soma += a[r];     // Entra o elemento novo.
    // Use soma: janela que termina em r.
}
```

Exige $1 \leq k \leq n$. Para contar valores distintos, mantenha frequências: de 0 para 1, entra um distinto; de 1 para 0, sai um distinto. O template `sliding_window.cpp` usa map e apaga chaves com frequência zero.

Janela variável que encolhe enquanto a soma é grande depende de monotonicidade; a regra usual exige valores não negativos. A janela de tamanho fixo acima também funciona com negativos.

6. Prefixos com map e greedy com heap

Contar subarrays de soma x, incluindo negativos e zero

Subarray é um trecho contíguo, sem pular elementos. Procure prefixos anteriores que completem a soma desejada.

```
map<long long, long long> freq;
freq[0] = 1; // Prefixo vazio: trechos que comecam no inicio.
long long soma = 0, resposta = 0;
for (long long valor : a) {
    soma += valor;
    resposta += freq[soma - x]; // Consulta prefixos ANTERIORES.
    ++freq[soma];             // Depois registra o atual.
}
```

Motivo: $\text{prefixo_atual} - \text{prefixo_anterior} = x$. Logo, buscamos $\text{prefixo_anterior} = \text{soma} - x$. Funciona com negativos e zero. Se registrar antes de consultar, pode contar o trecho vazio quando $x = 0$.

Custo com map: $O(n \log n)$, memória $O(n)$. Use long long para somas, frequências e resposta; confirme que todas as operações cabem nesse tipo.

Greedy + heap mínimo: manter saúde não negativa

```
priority_queue<long long, vector<long long>,
              greater<long long>> pq;
long long saude = 0;
for (long long valor : a) {
    saude += valor;
    pq.push(valor);
    if (saude < 0) {
        saude -= pq.top(); // Desfaz o valor mais prejudicial.
        pq.pop();
    }
}
cout << pq.size() << '\n';
```

Hipótese: a saúde começa em 0. Escolha uma subsequência na ordem original, maximize sua quantidade e mantenha a saúde não negativa depois de cada elemento escolhido.

Se a saúde ficar negativa, retirar o menor escolhido recupera mais saúde com uma única remoção. Isso preserva a chance de aceitar elementos futuros. Uma remoção basta, pois a saúde anterior era não negativa.

A heap decide quais valores descartar; a subsequência mantém a ordem original. Custo $O(n \log n)$, memória $O(n)$. Confira a justificativa completa no guia antes de adaptar a outro objetivo.

7. Pegadinhas e complexidade

Evite	Lembrete correto
<code>cin >> a, b;</code>	<code>cin >> a >> b;</code> lê os dois.
<code>cout << '\n';</code> ou <code>"\\n"</code>	<code>cout << '\n';</code> quebra a linha: barra invertida.
<code>int x = st.pop();</code>	<code>int x = st.top(); st.pop();</code>
<code>s[i] == 0</code> para o dígito zero	Use <code>s[i] == '0'</code> ; 0 é um número.
<code>vector<int> v; v[0] = 8;</code>	<code>vector<int> v(n)</code> cria posições; <code>push_back</code> acrescenta.
<code>st.top() == x && !st.empty()</code>	<code>!st.empty() && st.top() == x</code> protege o acesso.
<code>long long z = a * b;</code> com <code>int</code>	<code>1LL * a * b</code> evita overflow intermediário de <code>int</code> .
<code>ms.erase(x)</code> para uma cópia	<code>find + erase(it)</code> apaga uma só.
<code>m[x]</code> só para verificar existência	<code>m.find(x) != m.end()</code> não cria chave.
<code>v[v.size()]</code>	Último índice: <code>size() - 1</code> , se não estiver vazio.
Comparador com <code><=</code>	Use ordem estrita e consistente.
Ordenar antes de contar subarrays	Ordenar muda a vizinhança e os intervalos.

Complexidade: conferir o tamanho da entrada

Custo	Com n = 200.000	Exemplo
$O(1)$	Constante	Acesso a um índice
$O(\log n)$	Cerca de 18 níveis	Busca em árvore balanceada
$O(n)$	200 mil passos, estimativa	Uma passagem
$O(n \log n)$	Cerca de 3,6 milhões, estimativa	sort / operações de map
$O(n^2)$	40 bilhões, estimativa	Geralmente inviável para esse n

Os números ilustram crescimento; não medem tempo exato. Limite de tempo, máquina, custo por operação e memória também contam. Dois loops separados são $O(n)$; dois aninhados podem ser $O(n^2)$.

Na janela deslizante, se cada ponteiro avançar no máximo n vezes, o total pode continuar $O(n)$, mesmo com while dentro do for.

Checklist: entrada correta > casos mínimos > índices > estruturas vazias > overflow > custo > saída exatamente como pedida.